GSoC'26 Project Proposal

Sugar Labs: Sugarizer ConnectTheDots

Rejah Rabeeul Haque

Basic Details

Full Name:

Rejah Rabeeul Haque

Emails and Contacts:

Primary email: rejahrabeeul@gmail.com

Secondary Email: rabeeulrejah@gmail.com

Github: Rejah-Rabeeul

Discord: Rejah Rabeeul
Phone: (+91) 7592093288

Your First Language:

My first language is Malayalam but I am proficient in speaking, reading, writing, and understanding English.

Location and Timezone:

Location: Kozhikode, Kerala, India
Timezone: Indian Standard Time (UTC+5:30)

Communication:

The time I will be comfortable working with:

- UTC 22:30 – 02:00 (IST 04:00 – 07:30)
- UTC 08:00 – 12:00 (IST 13:30 – 17:30)
- UTC 16:30 – 19:00 (IST 22:00 – 00:30)

I can be flexible on my day 1 hours early or late if it helps to communicate with other developers and mentors, and I will be reachable anytime through my Mobile No, Discord and Email.

Education Details

I am currently in my third year of pursuing a Bachelor of Technology in Computer Science and Engineering from KMCT Institute of Emerging Technologies and Management under **APJ Abdul Kalam Technological University**. I have started programming in my second year, and I have gained a lot of experience building up my skills in Data Structures and Algorithms, Operating Systems, Cyber Security, and Web Development. I have also been able to apply my skills through various hackathons and projects within my home state throughout the years. Lately, I have focused on learning more about AI and Machine Learning currently, I developed an algorithm for predicting Road Accidents using historical records, weather, and road characteristics and time factors that contribute to where accidents may occur. I have begun to contribute to Open Source projects since January 2026, specifically with Sugar Labs (Sugarizer). In this project, I am developing enhancements for web-based tools for education. I have experience programming mainly in C/C++, Python, and new web application development, I am also an eSports professional, which has helped me build my teamwork capabilities outside of school.

Share links, if any, of your previous work on open source projects

I started my open source journey with Sugar Labs in my third year of undergraduate studies. After that, I learned more about the community, its structure, and the way people interact in this field.

Since then, I have been continuously contributing to Sugarizer by Sugar Labs for nearly 3 months. During this time, I have contributed to many activities such as Fototoon, Chess, Paint, Flip, Calculate, and FractionBounce — fixing bugs, adding enhancements, and implementing new features. I have also started working on my own activity, ColorLab (which is nearly complete), that helps kids learn about color theory. This has also given me a deeper understanding of how Sugarizer works internally.

Sugar Labs:

Pull request link	Description	Status
#1984	Created a activity ColorLab that helps kids to learn about color theory	Pending Merge

#1916	Added Resize feature on Fototoon activity	Merged
#1954	Bug in Fototoon Window resize	Merged
#2021	Enhanced Features on Fototoon activity Globes not displayed when exporting a single image in fototoon	Merged
#1948	Fix: In calculate activity Delete key deletes wrong character issue	Merged
#2041	Enhanced features on Chess activity Saved Chess game fails to restore difficulty setting	Merged
#2058	Fototoon "Clean All" leaves image data in storage while clearing the UI	Merged
#1966	Bug in trigonometric evaluation on Calculate activity	Merged
#2065	Fullscreen Reset Bug in FractionBounce Causes Ball Selection Loss & Bounce Glitch	Merged
#2075	Fraction activity user changes are not saved for ball and background	Merged

I have made over **27 commits** till now and will contribute more.
The most significant contribution was creating a activity ColorLab that helps kids to learn about color theory by interactive games. For this contribution I've added 16,511 lines.

I worked on some of my personal projects for learning purposes and I have also participated in National Cybersecurity Conclave and other hackthons.

Convince us that you will be a good fit for this project, by sharing links to your contribution to Sugar Labs

I am an active contributor of **Sugarizer**(Sugar Labs) since **Jan 2026** with over **27 commits** merged in various activities. Spending this much time with the codebase helped me to understand it in a better way.

- Commits: 27 (16,511++, 321--)
- Issues: 11 (10 closed, 1 open)
- Pull Request: 10(9 closed, 1 open)

I am applying to Sugarizer (Sugar Labs) for Google Summer of Code 2026 because of its strong community relations and the meaningful work carried out by this organization. Here, everyone discusses and develops as a team, and every opinion matters. It truly makes you feel at your best knowing that your work contributes to a social cause and creates a positive impact on society.

Project Details

What are you making

The aim of this project is to create a ConnectTheDots activity that matches with the Sugarizer environment, it will include 3 distinct mode such as Draw mode, Number mode and Game mode, so I will be using:

- **sugar-web/activity/activity** — to handle the activity lifecycle and to connect Stop button and other toolbar actions to the Sugarizer system.
- **sugar-web/datastore** — to save and restore full activity state and canvas images to the Sugar Journal.
- **sugar-web/graphics/presencepalette** — to handle joining the Sugarizer Presence Server so we can synchronize activity with multiple users.
- **sugar-web/graphics/palette** — for all toolbar dropdowns like color picker, mode selector, and template selector etc. Also, they auto-close when another palette opens, respond to keyboard navigation, and adapt visually to the user's active Sugar theme without any extra work on my end.
- **sugar-web/env** — to read the user's language and their Sugarizer profile color so the activity synchronizes just like other activities.
- **sugar-web/graphics/help —** to create tutorial for the activity
- **l10n.js —** to handle Localization. It searches for translations based on the user's env language preference.

For the win animations I am using canvas-confetti, which gives a fullscreen celebration when a child completes a Number Mode or wins a Game Mode session.

System Architecture

```
ConnectTheDots.activity/
├── index.html                  # The main page that shows the game
├── activity/                   # Tells Sugarizer the name and icon
│   ├── activity.info           # Settings for the activity
│   └── activity-icon.svg       # Activity icon
├── js/
│   ├── loader.js               # Sets up all the code files
│   ├── activity.js             # Handles Sugarizer (saving and networking)
│   ├── core.js                 # Code for drawing, resizing, and clicks
│   ├── modes/
│   │   ├── draw.js             # Logic for Draw Mode
│   │   ├── number.js           # Logic for Number Mode
│   │   └── game.js             # Logic for Game Mode (Includes AI)
├── css/activity.css            # The styling for the activity
├── icons/                      # SVG icons for toolbar buttons
├── locales/en.json...          # Language translation files
├── lib/                        # Standard Sugarizer tools and extra libraries
│   ├── sugar-web/              # Standard Sugarizer tools
│   ├── require.js              # Code file loader
│   ├── confetti.browser.min.js # Winning effect
│   ├── l10n.js                 # Localization support
│   └── ...                     # Other helping libraries
├── package.json                # Project definition file
├── LICENSE                     # Open-source license
└── NOTICE                      # Project notice file
```

Implementation

1. Grid and Canvas

In the activity all three modes share one canvas and the same underlying dot grid. The grid is a uniform array of evenly spaced points that covers the entire canvas area. I handle high-DPI screens by recalculating the canvas buffer using the device pixel ratio on every resize, while keeping the drawing coordinate system in CSS pixels so the mode logic never has to think about screen density. This means the activity renders sharply on any device without any special casing. Also, any grid dot near the user's cursor or finger grows slightly on every frame. This helps children get a clear visual cue about where their next line will snap, which makes the experience feel much more responsive on touch screens.

I am using a requestAnimationFrame for smooth visuals and a slow backup timer to keep the activity active in the background. This prevents the game from freezing when the browser tries to save battery or when the user switches to another app.

2. Toolbar and Palette

The toolbar follows the standard Sugarizer pattern. The activity button,network sharing button, and the right-aligned help, stop and fullscreen buttons are all wired to the Sugar-Web lifecycle automatically when the activity starts. For all the toolbar dropdowns the color picker, mode selector, and template selector I am using **sugar-web/graphics/palette** module. I because it gives the same behavioral guarantees as every other palette in Sugarizer. They auto-close when another palette opens, work correctly with keyboard navigation, and adapt visually to the user's Sugar theme.

Every time the user switches modes, I run a full teardown first — clearing all drawing state, lines, fills, and stopping any running process before initializing the new mode. This makes sure nothing leaks between modes. After the switch the toolbar automatically adjusts so only the relevant controls are active:

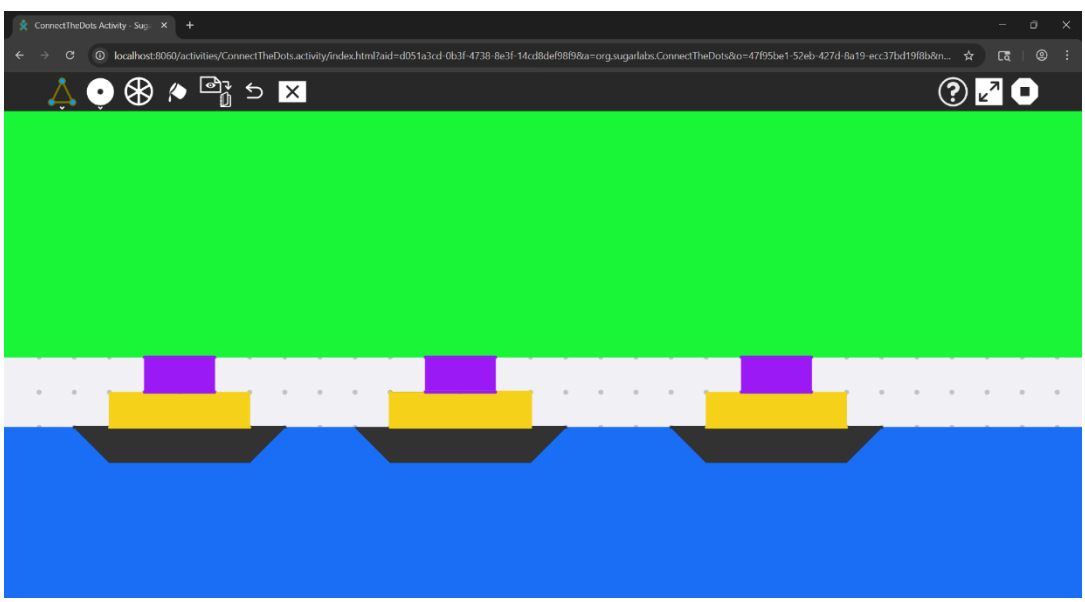
Button	**Role**	**Active**
Activity Button	Activity title & Journal settings	Always Active
Network Button	Sharing & Multiplayer	Always Active
Color Picker	Pick the active drawing color	Always Active
Mode Selector	Switch between Draw, Number, and Game Modes	Always Active
Template Selector	Pick a shape puzzle to solve (includes built-in and custom templates)	Active on Draw & Number Mode
Save as Template	Saves the dots as a playable puzzle that appears in the template palette	Active only on Number Mode
Save as Image	Export the current canvas as a PNG to the Journal	Always Active
AI Toggle	Auto-draw templates (Draw/Number Mode) or toggle computer opponent (Game Mode)	Always Active
Undo	Step back one move to fix a mistake	Active on Draw & Number Mode

Clear	Erase everything on the canvas	Always Active
Help Button	Shows a guide for how to play	Always Active
Fullscreen	Fills the entire screen with the canvas	Always Active
Stop	Saves gracefully and closes the activity	Always Active

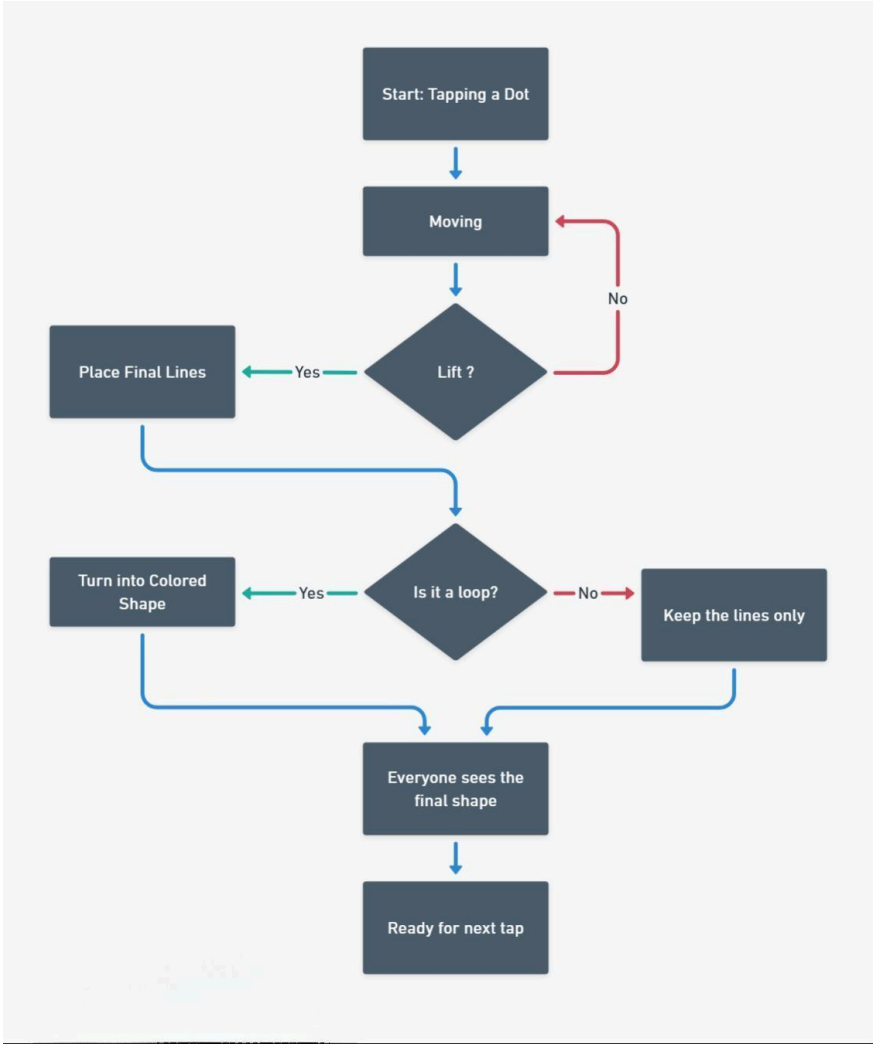
I’m building the toolbar using standard HTML5 and CSS3. Each button is created in the index.html file and then its logic is handled in js/activity.js. For the dropdown menus (like the color picker and mode selector), I use the official sugar-web/graphics/palette module. This ensures that every menu works smoothly.

3. <u>Draw mode</u>

Draw Mode is designed for the younger users. It is the mode where a child can just pick up and draw without needing to understand any rules. When they press on any empty grid intersection a dot appears and a drag starts. As they drag across the canvas, intermediate dots are automatically inserted at every grid intersection the path crosses, snapping cleanly to the grid regardless of the angle or speed of the drag. While dragging, a faint live ghost line follows the finger or cursor that shows exactly where the strokes will snap before the child commits the action. This provides instant visual feedback and makes the drawing feel incredibly responsive.



Flow Chart



Closing a shape and filling

when a drag returns to a dot that is already part of the current stroke, the activity detects a closed polygon and immediately fills the enclosed area with the currently selected color. This happens every single time an area is closed, so the child gets instant visual feedback as each shape completes.

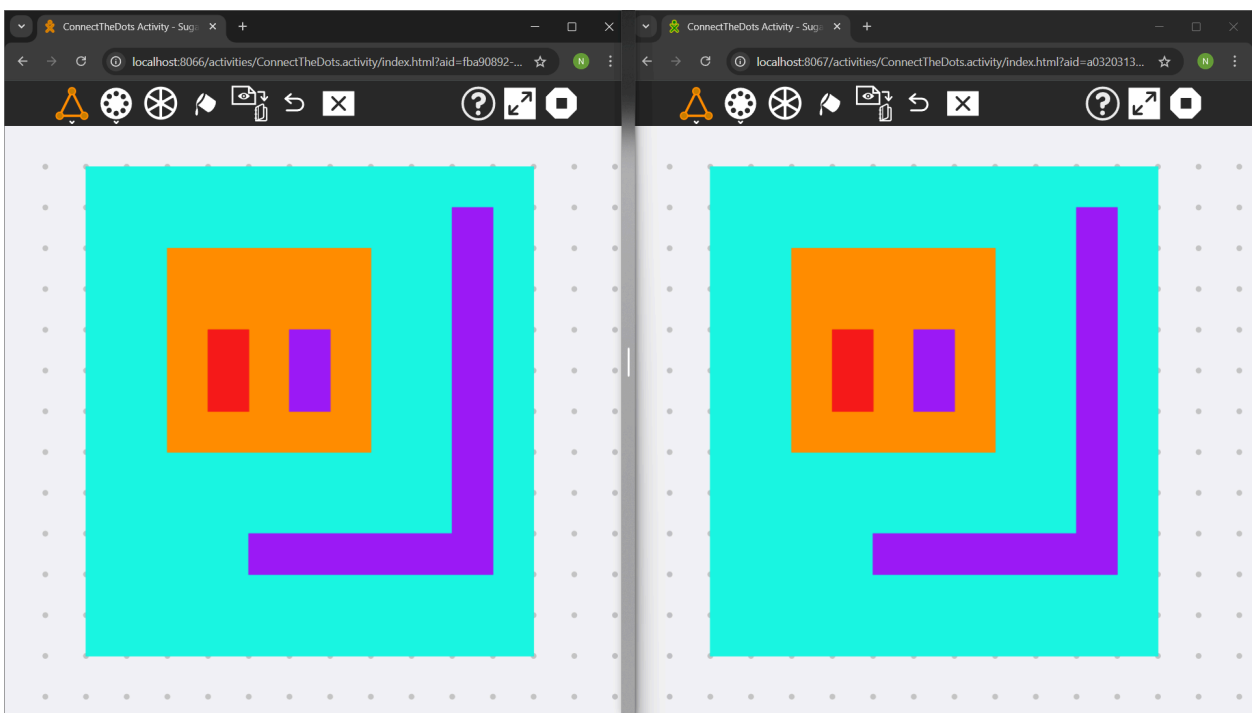
Color layering

The most important design decision in Draw Mode is how colors behave when shapes overlap. The rule is simple: every enclosed area always holds exactly the color it was filled with when it was first closed, and a new shape drawn on top never replaces or collapses those existing colors. Only the genuinely new area enclosed by the new shape gets the new color.

To make this work correctly, the fill system tracks all existing filled regions and their colors independently. When a new area is closed, before filling it the system checks every grid cell inside the new boundary. Cells that already belong to an existing filled region keep their current color and are not repainted. Only cells that are not yet part of any fill receive the new color. This means colors never bleed into each other and a child can freely draw overlapping and nested shapes without any unexpected color changes. The boundary lines of a completed fill are hidden automatically so the canvas looks clean rather than showing both the outline and the fill on top of each other.

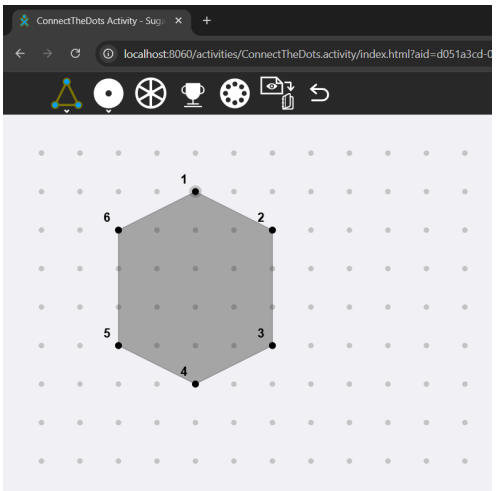
Shared Mode

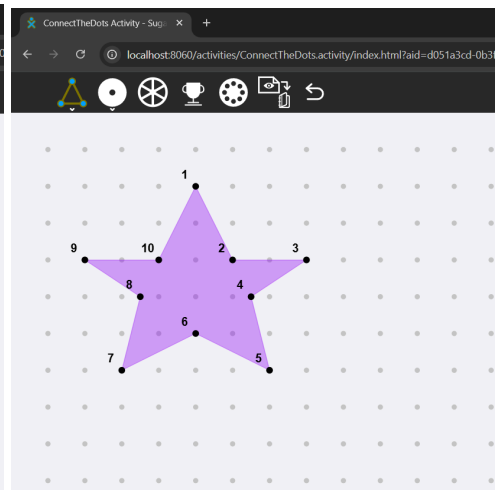
Shared Mode allows multiple children to draw and interact on the same canvas in real time. It provides the same functionalities as Single Mode, with all actions instantly synchronized so everyone can see changes as they happen. This creates a simple and collaborative experience where children can learn and create together.



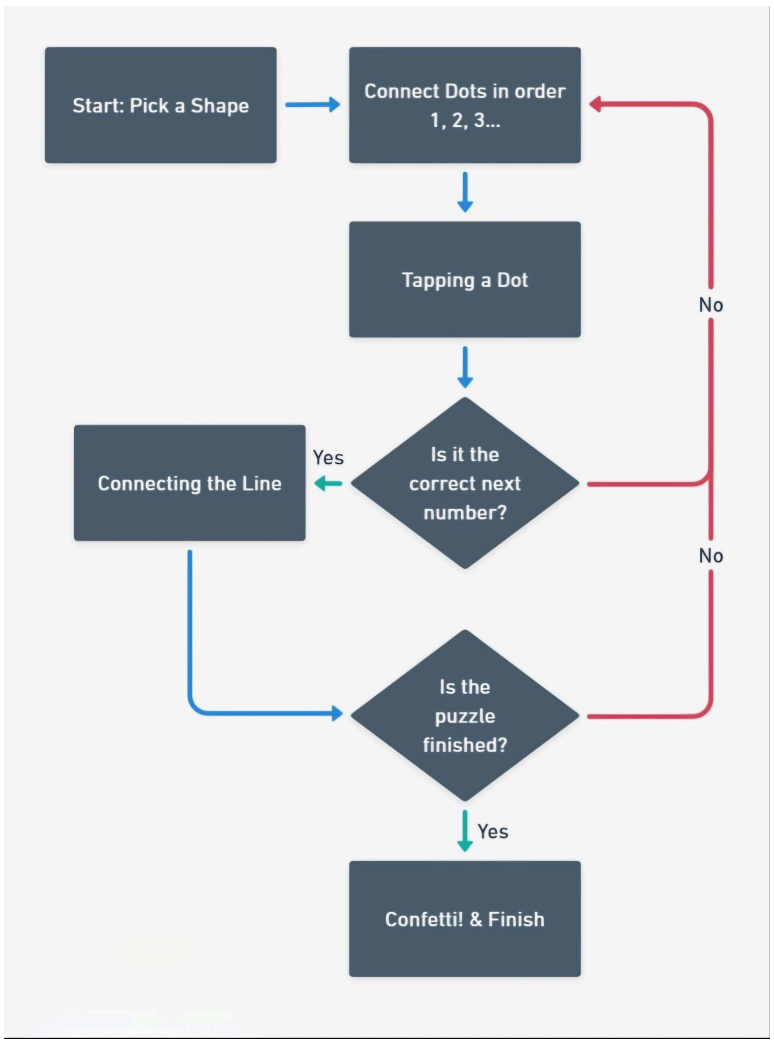
4. Number mode

Number Mode is the structured learning mode. The child sees a set of numbered dots on the grid and has to connect them in the right order to reveal a hidden picture, just like a classic connect-the-dots puzzle.


ConnectTheDots Activity - Sug
localhost:8060/activities/ConnectTheDots.activity/index.html?aid=d051a3cd-0
1
2
3
4
5
6


ConnectTheDots Activity - Sug
localhost:8060/activities/ConnectTheDots.activity/index.html?aid=d051a3cd-0b3
1
2
3
4
5
6
7
8
9
10

Flow Chart


Start: Pick a Shape
Connect Dots in order 1, 2, 3...
Tapping a Dot
Is it the correct next number?
Yes
Connecting the Line
No
Is the puzzle finished?
No
Yes
Confetti! & Finish

Built-in Template

The activity will include several built-in templates. This includes Numerical Figures (learning to connect dots to form digits 0-9), Mathematical Shapes (like a Hexagon, Star, etc), and Creative Designs for younger children, such as a Heart, Tree, House, Rocket, and a Boat.

Custom Template

The template editor lets children design their own puzzles by placing dots on the grid and then tapping them in the sequence they should be numbered. The "Save" button only becomes active once the child has drawn a valid, closed shape. This ensures that every puzzle written to the Journal is complete and playable, removing any risk of a child saving a half-finished drawing by mistake. Once the shape is ready, it is saved as a Journal entry tagged with a unique MIME type so it can be automatically loaded into future sessions.

Journal Integration and Error Handling

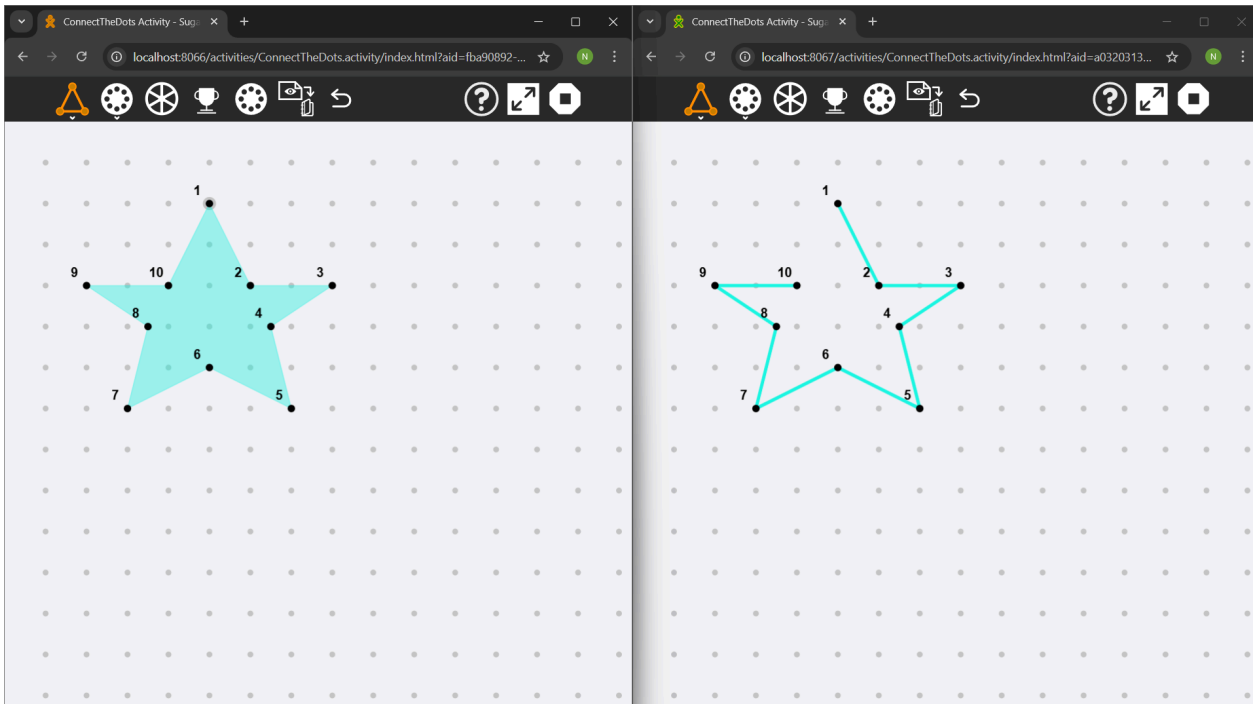
In Number Mode, when the activity starts, it uses the Sugar-Web Datastore to search the Journal for any custom puzzles previously saved by the child. It performs a background search using datastore.find() and automatically adds these puzzles to the palette, so the child does not need to import anything manually.
The dot-connecting rules are designed to be stress-free. If a child tries to connect a dot that is not next in the sequence, the activity simply ignores the action.

This follows a "silent feedback" approach, avoiding loud error sounds or distracting visual warnings. Such feedback can be discouraging for young children. Instead, the activity gently encourages them to observe the numbers and try again independently.

Once the final dot is connected, the shape is immediately filled with the selected color, followed by a confetti celebration on the canvas.

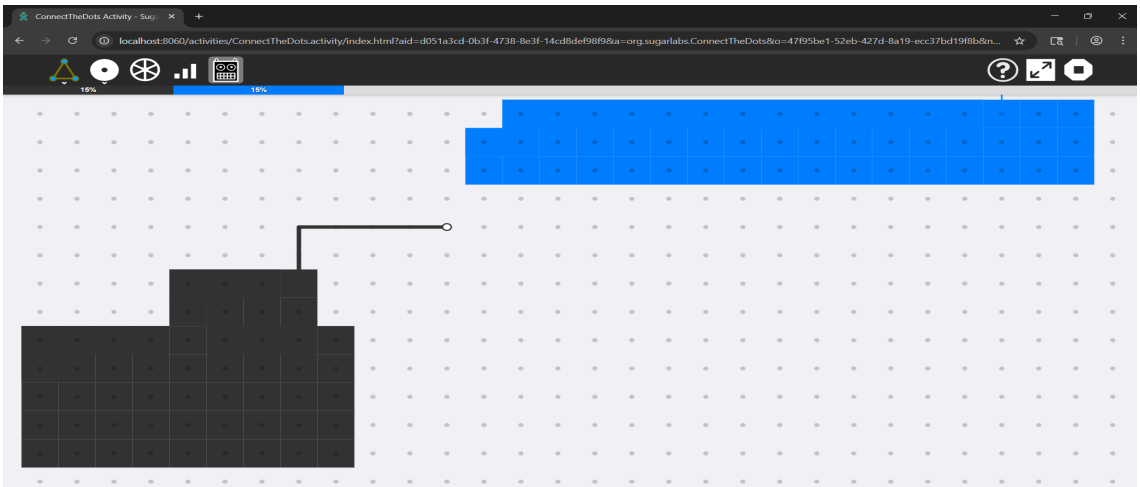
Shared Mode

In shared mode, all connected players race to complete the same template. The first to correctly connect every dot wins. A child can also share a template they created, pushing it to all peers in the session so everyone plays the same puzzle.

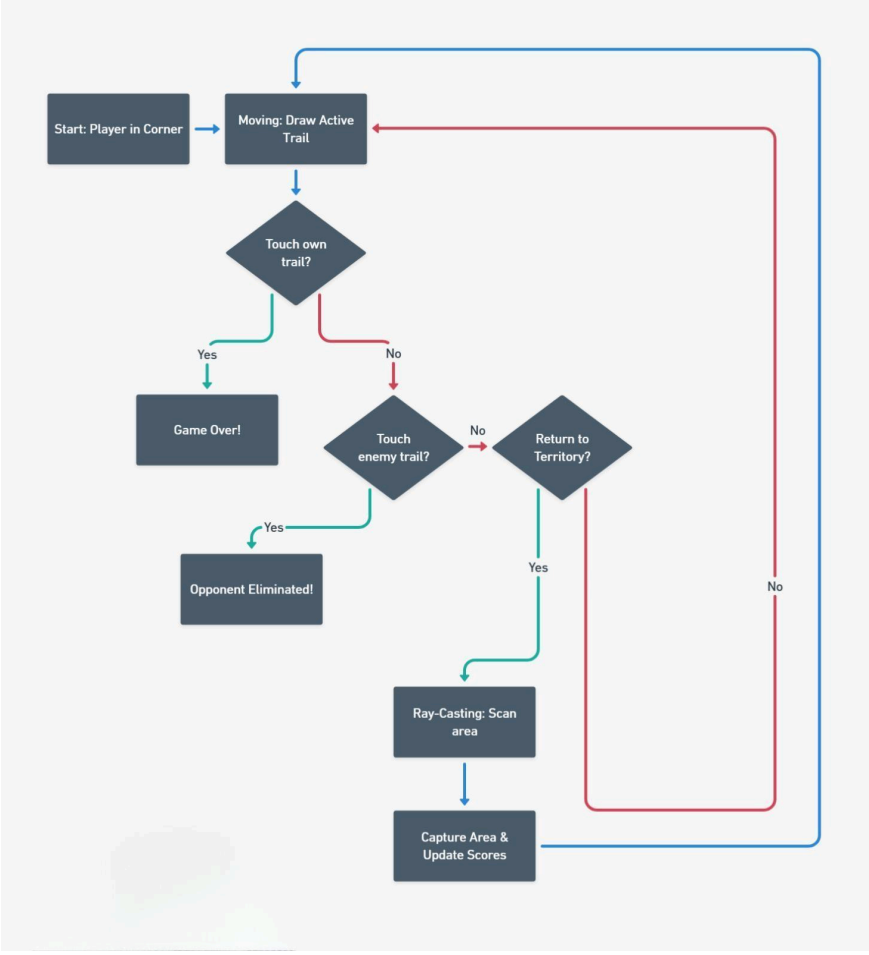


5. <u>Game mode</u>

Game Mode is the competitive mode, inspired by paper.io. Player start with a small block of territory in a corner of the grid — the local player gets one corner, the AI gets the opposite corner, and any additional network players fill the remaining areas in the order they join.



Flow Diagram



Movements

player moves one grid step at a time using arrow keys or swipe gestures. The movement system enforces three rules: you cannot go outside the canvas, you cannot immediately reverse direction along your own trail, and if your trail head touches your own trail you are eliminated. If your trail head crosses an opponent's active trail, that opponent is eliminated instead.

Territory capture

When a player's trail head returns to their own territory, everything enclosed by the loop they just drew gets captured. To figure out which grid cells are inside that loop I use the **even-odd Ray Casting** algorithm. For each candidate cell I cast a horizontal ray and count how many times it crosses the loop boundary; an odd count means the cell is genuinely inside. I chose Ray Casting over a simple flood fill because it handles any loop shape correctly, including non-convex and concave ones. A child can draw a star-shaped loop or an L-shaped loop and the algorithm will capture only what is truly inside, without leaking through corners or gaps. Cells

belonging to an opponent that fall inside the loop are stolen, and an opponent who loses all their territory is eliminated immediately.

AI OPPONENT

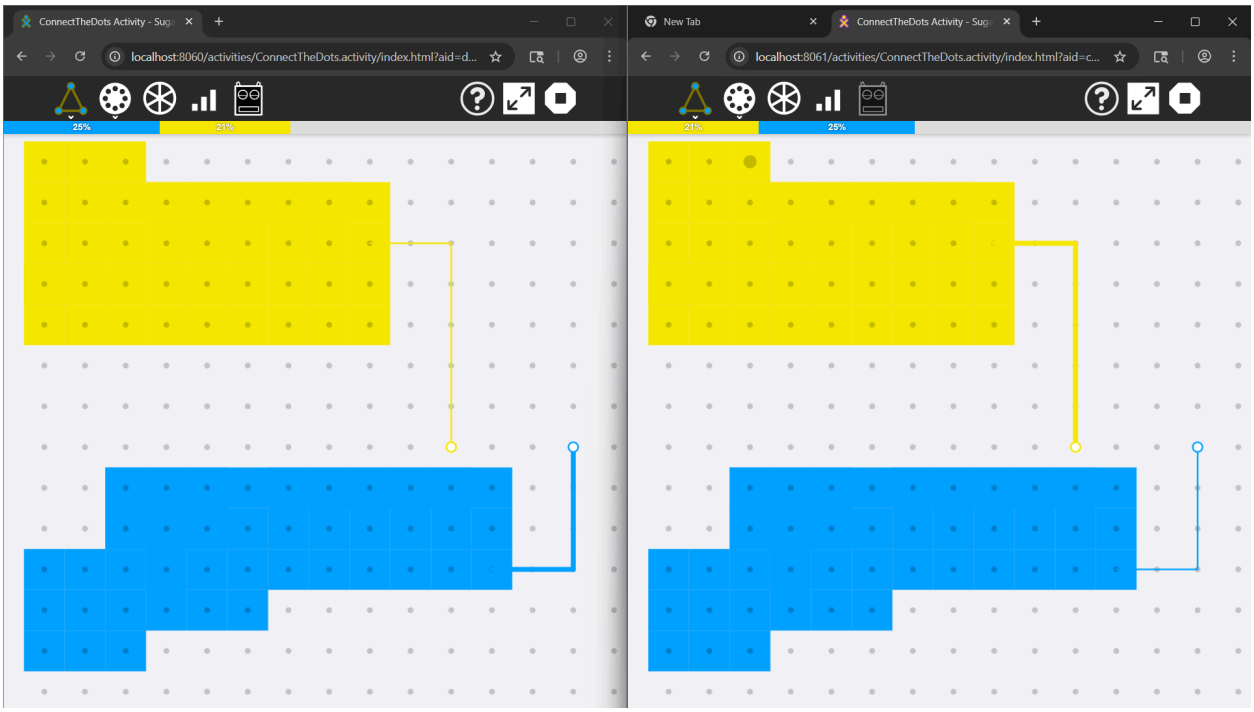
I will implement a proper scoring greedy heuristic for the AI opponent. At each update cycle, the AI will enumerate all loop-closing paths it can reach within a bounded depth-first search, score each one by how many cells it would capture using the same Ray Casting function, and execute the best option. I will also add a noise factor for the easy difficulty so the AI occasionally picks a slightly worse move — this makes it feel like a fair opponent for a 5 to 11 year old rather than an unbeatable machine. The AI toggle appears in Game Mode and in shared sessions the host can turn it on or off.

Scoring

Below the toolbar there is a live progress bar divided by player color showing what percentage of the total grid each player owns. It updates every frame so the tension of the game is always visible. When only one human player remains, the session ends with a confetti animation.

Shared Mode

In shared mode, each player gets a unique color when they join, and we will run a conflict check automatically so no two players share the same color. Movement, trail updates, and territory captures sync to every peer in real time. For conflict resolution, I will use a Dynamic Overwrite system: the board simply updates based on the move that arrived most recently. This allows players to naturally take territory from each other, making the game more fast-paced without needing complex timestamps or host authority.



Challenges and Mitigations

Multiplayer territory conflicts and overlaps

Rather than using complex timestamps, I will implement a "Dynamic Overwrite" system. On every device, the most recent message received simply updates the ownership of those cells. This makes the game feel fast and competitive, as players can naturally steal territory back and forth without any lag-inducing checks.

Performance on low-end

To ensure smooth gameplay on older hardware, I am using a flat `Uint8Array` instead of coordinate objects. Iterating over 600 bytes of typed data with direct index reads is significantly faster and uses much less memory.

Template discovery across sessions

Custom templates will be saved as Journal entries tagged with the activity's unique MIME type. On startup, the activity queries the Journal for these entries and automatically adds them to the template palette alongside built-in puzzles.

Rejoining a shared session

When a player disconnects, I will have the host broadcast a complete state snapshot to any rejoining player. This allows the client to quickly replace its local state and stay perfectly in sync.

Localization gaps

Every piece of text—from labels to win screens—will be moved into locale JSON files. Using the l10n module with a fallback strategy (English as the base) ensures the activity is fully usable in French and Spanish.

How will it impact Sugar Labs?

Sugarizer is a great platform with many features. It helps many students across country to learn things by playing games also ensuring they are not getting addicted by ensuring minimal UI designs. Currently, Sugarizer has so many activities that can help kids to learn many fields by including Connect the Dots activity the biggest differentiator is that no other Sugarizer activity currently covers all three of creativity, structured learning, and strategic gameplay. This ensures that kids aged 5-11 can feel enthusiasm. The AI opponent in Game mode is also entirely new for Sugarizer most shared activities are purely human-vs-human. Adding a bot means a child can practice strategy even without network peers.

What technologies (programming languages, etc.) will you be using?

ConnectTheDots is a simple web application written entirely on Vanilla JS, HTML5, and CSS3, with its core engine relying completely on the HTML5 Canvas 2D API.
ConnectTheDots will completely built by integrate with the Sugar-Web framework, making use of its built in modules. Such as sugar-web/datastore for saving user progress to the Journal, sugar-web/env for matching the user's Sugarizer theme and language, Real-time multiplayer functionality by the Sugarizer Presence API.
On the technical side, ConnectTheDots includes several algorithms such as, Breadth First Search (BFS) for shape detection, Ray Casting for territory calculation, and Greedy Heuristics to drive the AI opponent. The application also integrates supporting libraries such as i18next for multilingual localization, with support for English, French, and Spanish.

Timeline

Break down the entire project into chunks and tell us what will you work on each week.

Days	Tasks
Pre GSoC Current date to May 1, 2026	➢ Completing the ColorLab activity. ➢ Stay connected with the community and contribute to Sugarizer.

Community Bonding Period May 1, 2026 to May 24, 2026	➢ Discuss the creation of the ConnectTheDots activity and learn about it ➢ Discuss about the modes that are going to be in the activity ➢ Also, discuss about how each mode should behave on the shared mode ➢ Explore the tech stack required for the project.
Week 1 May 25, 2026 to May 31, 2026	➢ Create the complete production folder and file structure with all modules separated. ➢ Build the production Canvas engine as a pure rendering module. ➢ Implement responsive dot grid rendering that correctly recalculates dot positions and spacing on window size. ➢ Implement a dirty region render loop so only changed portions of the Canvas repaint each frame.
Week 2 Jun 1, 2026 to Jun 7, 2026	➢ Build the unified input abstraction layer handling mouse and touch events identically. ➢ Implement dot hit detection with a comfortable tolerance radius. ➢ Add hover over the dots and a ghost line that follows the cursor between dots before commits a connection. ➢ Implement line rendering with clean antialiasing and consistent stroke width across all screen densities. ➢ Develop Draw Mode as the first fully playable mode.
Week 3 Jun 8, 2026 to Jun 14, 2026	➢ Implement Number Mode as the second game mode, built using the same Canvas engine and input system. ➢ Number labels are rendered on each dot the user must connect them strictly in ascending numerical order to see the picture also out-of-order connections are rejected ➢ Render the completed drawing progressively as correct connections are made, revealing the hidden picture line by line ➢ Track completion state and display a congratulations confetti animation screen showing time taken when all dots are connected correctly ➢ Develop default set of built-in templates

Week 4 Jun 15, 2026 to Jun 21, 2026	➢ Build the template editor : a canvas where users can place dots freely, assign numbers to define the correct sequence, preview the resulting drawing, and save it as a custom template ➢ Implement template save to sugar-web/datastore as a Journal entry ➢ Build the template gallery UI : a browsable view for all default templates and saved templates in the Journal that users can select and play ➢ Implement template sharing via the Presence API so users can push their custom templates to all peers in a shared session by mirroring how Calligra and Tangram handle shared content ➢ Validate the functionality
Week 5 Jun 22, 2026 to Jun 28, 2026	➢ Implement the paper.io inspired territory expansion mechanic each player starts from a small square and expands their territory by drawing a path that forms a closed loop back to their territory. ➢ When a loop is completed, run BFS(Breadth First Search) flood fill to capture all cells enclosed within the player's territory. ➢ Use Ray Casting to accurately determine which cells are inside the completed loop for complex paths. ➢ Handle the edge case scenario where a player's path crosses their own trail , another user trail or ai's trail
Week 6 Jun 29, 2026 to Jul 4, 2026	➢ Build the Greedy Heuristic AI that evaluates all possible loop completions, scores each by territory area gained, and executes the best option, introduces easy difficulty to feel beatable ➢ Integrate the AI automatically when a player starts Game Mode alone ➢ Implement the scoring system display as a live progress bar for total grid owned by each player ➢ Integrate sugar-web/activity lifecycle and sugar-web/datastore to save and restore the complete game state for all three modes including Draw Mode canvases, Number Mode progress, and Game Mode territory maps ➢ Build the Sugar toolbar with Reset, Mode Switcher, Network, Color and a Robot icon for turning on and off AI also common icons such as full screen, tutorial, stop, activity icon

Phase 1 Evaluation Jul 5, 2026 to Jul 10, 2026	➢ Midterm deliverable: All three modes fully playable with Sugar-Web integration complete.
Week 7 Jul 5, 2026 to Jul 11, 2026	➢ Integrate the Sugarizer Presence API with WebSocket-based session management ➢ Draw Mode multiplayer: all connected players draw on a shared canvas simultaneously every line placed by any player is broadcast and rendered on all peer’s screens in real time ➢ Number Mode multiplayer: All connected players compete to complete the same template simultaneously the first player to correctly connect all dots wins, also custom templates can be shared ➢ Game Mode multiplayer: Turn based territory conquest with all player positions and territory states synced. Also includes there will be an AI toggle for AI opponent ➢ Each player gets their own unique colour for territory players can change their colour but cannot select a colour already chosen by another player.
Week 8 Jul 12, 2026 to Jul 18, 2026	➢ Test all three multiplayer modes on a live Sugarizer server with real devices ➢ Fix state difference: Ensuring that the host's state is authoritative in all conflict scenarios ➢ Handle disconnection in each mode: Draw Mode pauses collaborative drawing, Number Mode continues with rest of the players, Game Mode also continues with rest of the players ➢ On rejoin, immediately push the full current state to the reconnecting player so they are back in sync ➢ Edge case testing: three or more players in all modes, host leaving mid game, slow or intermittent network

Week 9 Jul 19, 2026 to Jul 25, 2026	➢ Integrate i18next and allow it to dynamically read the current language from sugar-web/env. ➢ Create full locale JSON files in English, French, and Spanish for every string in the activity. ➢ Find all hardcoded strings in both the .js and .html files and replace them with i18next.t() function calls. ➢ Test the application in all three languages at runtime to ensure there are no layout issues and no missing text for any key. ➢ Implement a fallback chain so that any missing French (fr) or Spanish (es) keys fall back to English (en).
Week 10 Jul 26, 2026 to Aug 1, 2026	➢ Enhance UI/UX for every mode ➢ Polish all end game screens i.e., completion time and ranking for Number Mode, territory breakdown for Game Mode ➢ Refine the template editor UX by making dot placement and numbering intuitive enough for a child to use without instructions ➢ Ensure the full UI works correctly in both portrait and landscape tablet orientations ➢ Address all UI feedback from the mentor review at the end of Week
Week 11 Aug 2, 2026 to Aug 8, 2026	➢ Full end-to-end test of every flow : Draw Mode solo and shared, Number Mode solo and multiplayer with template sharing, Game Mode vs AI and multiplayer, template creation and Journal save/load ➢ Test in various browsers ➢ Fix all Canvas rendering bugs like dot alignment, line antialiasing, territory fill edges, color bleed between adjacent territories ➢ optimize BFS flood-fill and Ray-Casting to stay within frame budget on large grids ➢ Verify Journal save/load correctly restores all three modes including Draw Mode canvas state, Number Mode progress, and Game Mode territory maps ➢ Confirm all three locales display correctly with no errors ➢ Address all remaining mentor feedback from Week 10

Week 12 Aug 9, 2026 to Aug 15, 2026	➢ Final mentor review, address the last round of feedback and clean up remaining TODOs ➢ Cleanup and wrap up the work ➢ Complete all kinds of documentation ➢ Perform final testing and bug fixes to ensure stability ➢ Prepare the project for submission (code cleanup, final commits, and PR updates) ➢ Submit the final work product(ConnectTheDots activity) on the GSoC portal before August 24
Final Phase Aug 16, 2026 to Aug 24, 2026	➢ Every promised deliverable outlined in the proposal has been built, tested, documented and reviewed by the mentors. All feedback has been addressed the codebase is clean and merge-ready and the pull request has been submitted to the Sugarizer repository.

Following this timeline, I will be able to submit my final work product with extra features before the deadline of August 17, 2026. I am fully capable of allocating additional hours and delivering the work without breaking the existing timeline. Therefore, even in the worst case I am confident that I will be able to complete the project before the deadline.

How many hours will you spend each week on your project?

My college summer vacations start from May 13 to July 1. In this period I can give about 15-23 hours per week and after college resumes, I can give about 14-21 hours per week. I do not have any other commitments during the summer vacation and can fully devote my time to GSoC.

How will you report progress between evaluations?

I will be active on GitHub as well as Discord I will make regular pull requests to the Sugarizer while interacting with the mentors, so anyone in the org can view my progress. Thus, my progress will always be reported thoroughly on GitHub. I will also update all the features that is adding to the project to the mentor through Discord. I will be reachable anytime through Discord, Email, or a planned video session.

Why Me

I started my open source journey with Sugar Labs in my third year of undergraduate studies, gradually learning the community structure, contribution workflow, and how experienced developers collaborate in the open source ecosystem.

Since then, I have been continuously contributing to Sugarizer for nearly three months, working across multiple activities including Fototoon, Chess, Paint, Flip, Calculate, and FractionBounce fixing bugs,

adding enhancements, and implementing new features with over 9 merged pull requests and 27 commits so far.Beyond bug fixes, I have also built my own activity from scratch — ColorLab, which helps kids learn color theory through interactive games and is nearly ready to merge. Building a complete activity independently gave me a deep familiarity with the Sugar-Web framework, the Journal system, the Presence API, and the community review process. This experience is direct evidence that I have everything needed to build the ConnectTheDots activity to a production-ready, kids-friendly standard well within the GSoC timeline.

Discuss your post-GSoC plans. Will you continue contributing to Sugar Labs after GSOC ends?

I am planning to continue working on the Sugarizer and MusicBlocks(Sugar Labs) after the program ends. During my OpenSource journey Sugarizer was the first that got my eye so I would love to be a support system to the community also I would like to explore the MusicBlocks and give contribution to there also. I am amazed by the community relations and the work carried out by this organization. I got excited by watching the admin keeps getting active daily via Github and Discord with patience just to make the community stronger. I really want to enhance my skills and put them to use to give back to the community. I aim to develop mentorship skills and the ability to guide others and try to give back to the community by mentoring and guiding others. I hope to mentor future GSoC students.

Demo Links

- ConnectTheDots Working Prototype : https://youtu.be/0A2rWK3g0NI
- Pawn Activity : https://youtu.be/k9LoIOkyZjc

I am looking forward to contributing to Sugar Labs this summer season.
Kind Regards.